



ORQUESTANDO TAREAS CON AIRFLOW: TUS DATOS EN PILOTO AUTOMÁTICO

Sesión 1: Introducción a Apache Airflow

Vanessa Pradas Fernández | Software Data Engineer



00_

¿De dónde vengo?





Vanessa Pradas Fernández

- Me gradué en Ingeniería del software en la ETSII.
- Trabajo como ingeniera de datos en Next desde abril de 2025.
- Practico bjj, boxeo, canto y bachata.
- Mi linkedin [here!](#)

+140 especialistas

ÁREAS



Cloud



Data & AI



Analítica
Avanzada



Desarrollo de
Software

SECTORES

TRANSPORTE



LOGÍSTICA



TELECOMUNICACIONES



SEGUROS

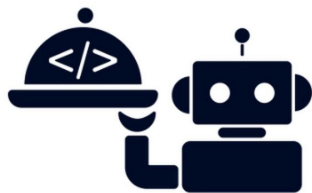


CONSTRUCCIÓN



ENERGÍA





SirviendoCódigo.com

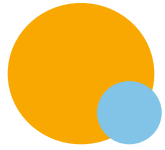
www.youtube.com/@SirviendoCódigo



Índice

- 1 ¿Qué es Airflow?
- 2 Servicios de integración
- 3 Componentes principales
- 4 Primer vistazo general
- 5 Toma de contacto



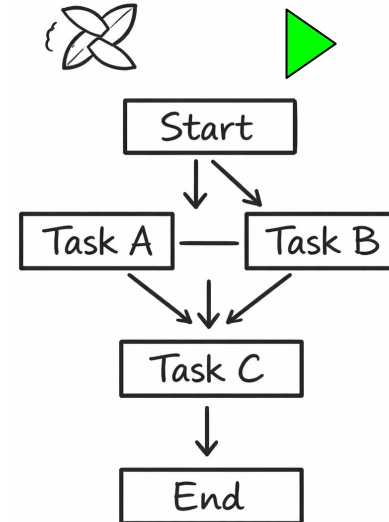
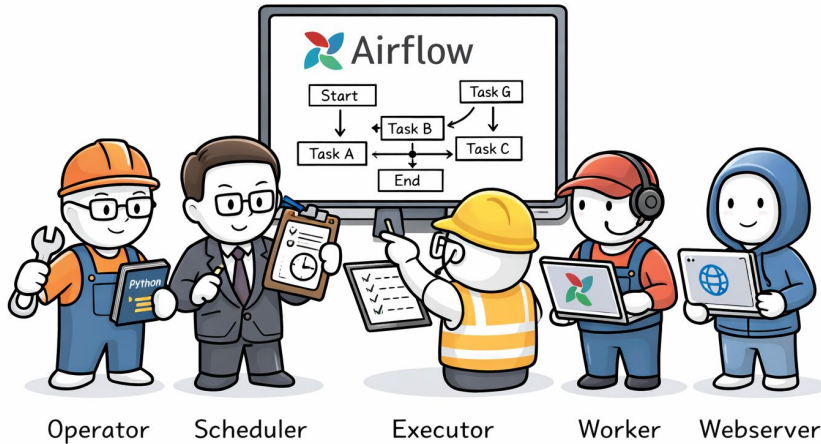


Objetivos



Entender qué es Airflow y sus componentes principales

Crear y ejecutar un DAG sencillo desde la interfaz de Airflow





01_

¿Qué es Airflow?

Airflow



Extract



Transform



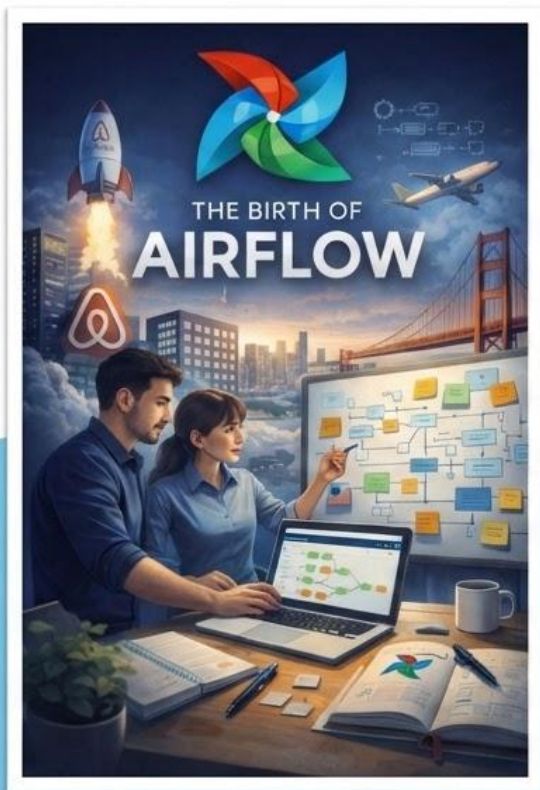
Load

XCom





Historia de Airflow



Desarrollada como herramienta interna de Airbnb



Donado como proyecto open source a la Apache Foundation bajo el nombre de Apache Airflow



En funcionamiento desde octubre de 2014



Objetivo: gestionar los ETLs de la compañía, cada vez más complejos



En ese momento las herramientas no eran escalables ni fáciles de mantener



¿Qué es Airflow?



Plataforma
open-source de
gestión de flujos
de trabajo



Automatizar,
programar y
monitorizar



Escalable,
dinámico,
extensible



Escrito en
Python



02_ Servicios de integración



Servicios de integración



BD
relacionales



BD no
relacionales



Data
Warehouses



Nube y servicios
gestionados





Servicios de integración



databricks



HIVE



APACHE Spark

Big Data y servicios de análisis



Airbyte



APACHE kafka



dbt

ETL e Ingeniería de datos



docker



kubernetes

Contenedores y servicios de orquestación



GitHub



GitLab



Jenkins

Control de versiones y CI/CD



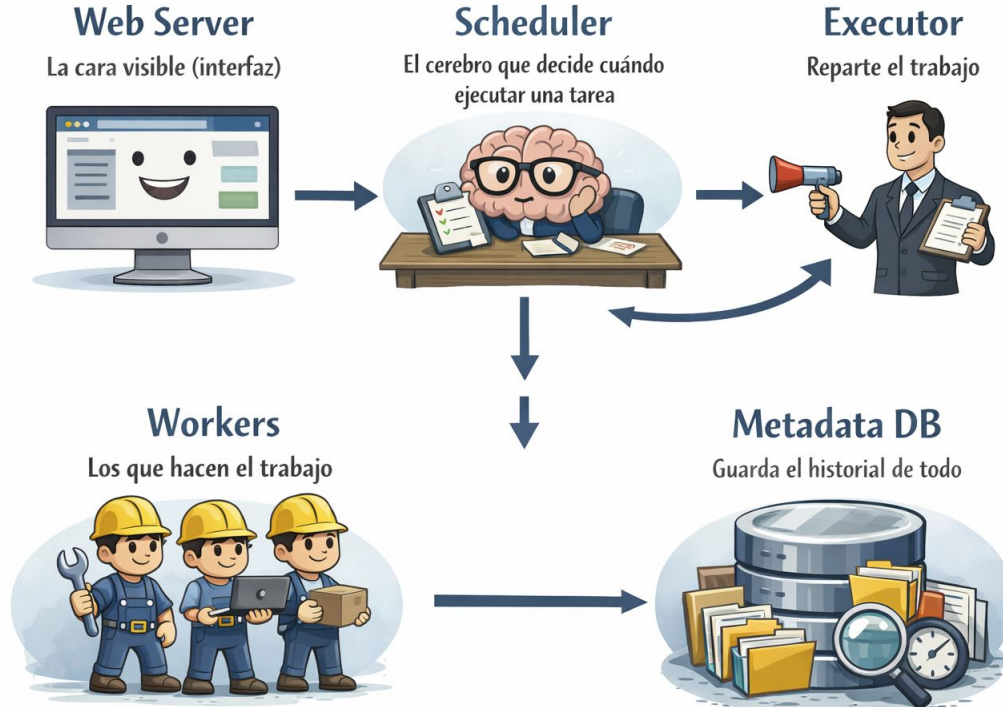


03_

Componentes principales

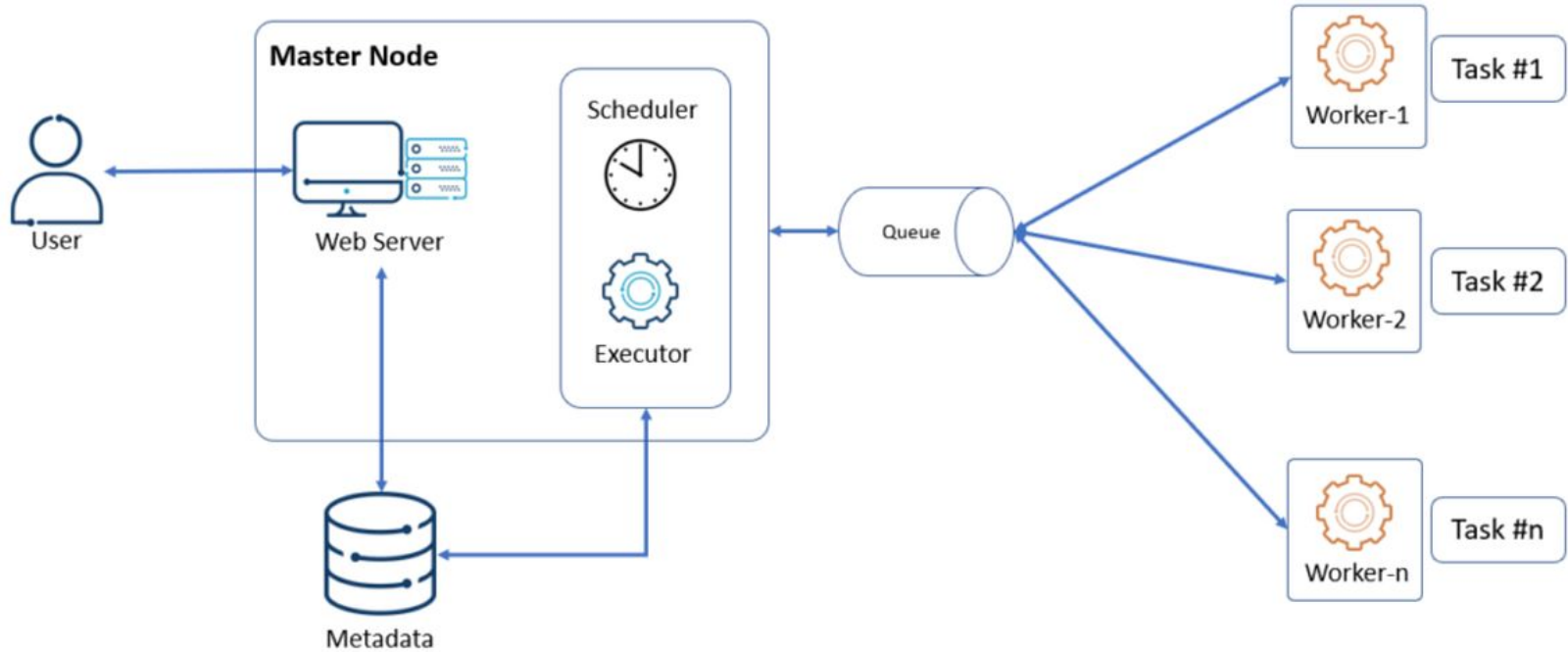


Componentes principales





Componentes principales





¿Qué es un DAG? **Directed** Acyclic Graph



Directed

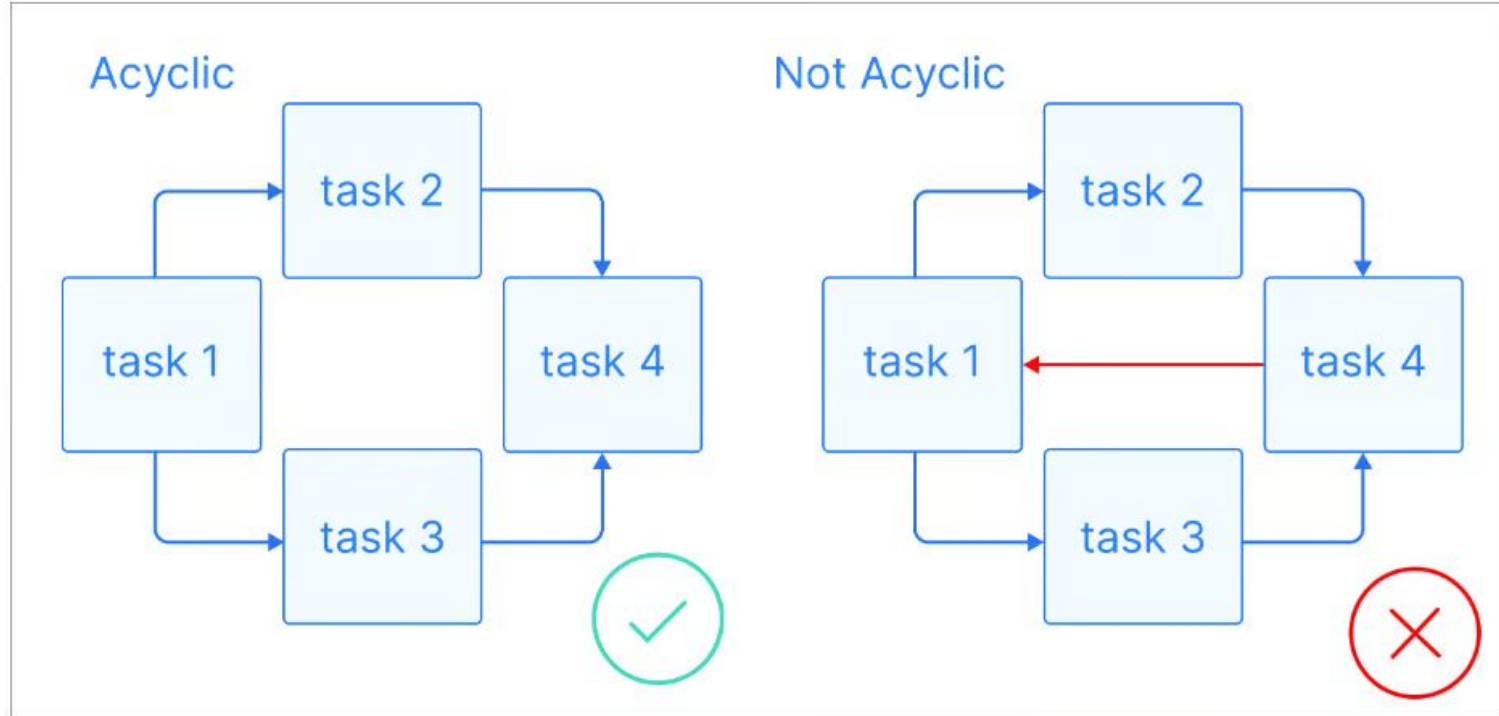


Not directed





¿Qué es un DAG? Directed **Acyclic** Graph





04_ Primer vistazo



Primer vistazo - UI



Home

Dags

Assets

Browse

Admin

Docs

User

DagsRunsTask Instances

Q Search Dags

Advanced SearchCtrl+K

AllFailedRunningSuccessAll

Filter by tag

2 Dags

Sort by Display Name (A-Z)

ejemplo_dag_pipelinedemo, pipeline

ScheduleLatest RunNext Run

0 0 * * *

2026-02-02, 01:00:00

example_astronautsspace, example

ScheduleLatest RunNext Run

0 0 * * *

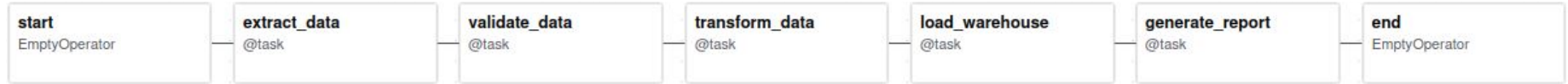
2026-02-02, 01:00:00

2026-02-03, 01:00:00

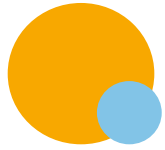
21



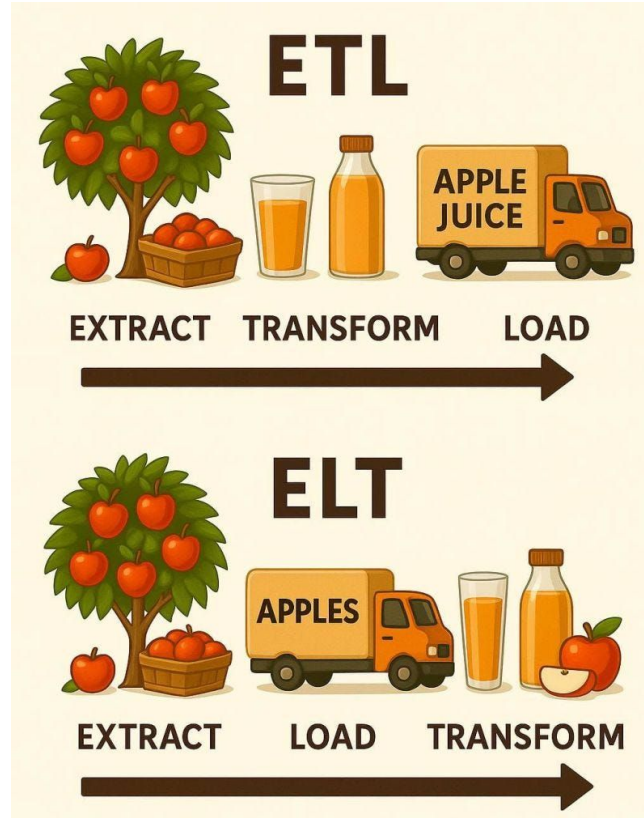
Primer vistazo - El dag



Workflow



Enfoque ETL vs ELT



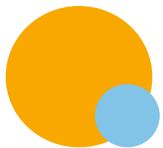


Primer vistazo - Decoradores



```
@dag(  
    start_date=datetime(2025, 4, 1),  
    schedule="@daily",  
    max_consecutive_failed_dag_runs=5,  
    doc_md=__doc__,  
    default_args={  
        "owner": "Astro",  
        "retries": 3,  
        "retry_delay": duration(seconds=5),  
    },  
    tags=["example", "space"],  
    is_paused_upon_creation=False,  
)
```

```
@task_group(group_id="proceso_limpieza")  
def seccion_limpieza():  
    @task  
    def borrar_nulos():  
        return "Nulos borrados"  
  
    @task  
    def normalizar_fechas():  
        return "Fechas corregidas"  
  
    borrar_nulos() >> normalizar_fechas()  
  
@task  
def enviar_reporte():  
    print("Proceso terminado")
```



Primer vistazo - Sin Decoradores



```
### ETL PROCESS ###

def extract_data(ti=None, **kwargs):
    raw_data = "Raw data"
    ti.xcom_push(key="raw_data", value=raw_data)

def transform_data(ti=None, **kwargs):
    raw_data = ti.xcom_pull(task_ids="extract", key="raw_data")
    processed_data = f"Transformed data: {raw_data}"
    ti.xcom_push(key="processed_data", value=processed_data)

dag = DAG(
    dag_id="my_first_dag",
    start_date=datetime(2026, 1, 1),
    schedule_interval="30 20 * * *",
    catchup=False
)
```

DAGs anteriormente (sin decoradores)



Primer vistazo - Operadores



a. PythonOperator (ahora decorator @task)

Transformaciones ligeras, validaciones y lógica condicional.



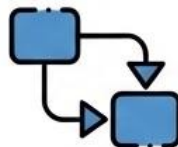
b. BashOperator

Ejecución de comandos o scripts.



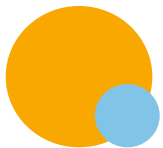
c. EmptyOperator (antes DummyOperator)

Creación de puntos de control, sin agregar funcionalidad.



d. >>

Define la dependencia entre dos tareas y controla el orden de ejecución.



Primer vistazo - Operadores



- Operadores más destacados de proveedores:
 - *SQL*

<code>BaseSQLOperator</code>	This is a base class for generic SQL Operator to get a DB Hook.
<code>SQLExecuteQueryOperator</code>	Executes SQL code in a specific database.
<code>SQLColumnCheckOperator</code>	Performs one or more of the templated checks in the column_checks dictionary.
<code>SQLTableCheckOperator</code>	Performs one or more of the checks provided in the checks dictionary.
<code>SQLCheckOperator</code>	Performs checks against a db.
<code>SQLValueCheckOperator</code>	Performs a simple value check using sql code.
<code>SQLIntervalCheckOperator</code>	Check that metrics given as SQL expressions are within tolerance of the ones from days_back before.
<code>SQLThresholdCheckOperator</code>	Performs a value check using sql code against a minimum threshold and a maximum threshold.
<code>BranchSQLOperator</code>	Allows a DAG to "branch" or follow a specified path based on the results of a SQL query.
<code>SQLInsertRowsOperator</code>	Insert rows (e.g. a collection of tuples) into a database table directly from an XCom or Python data structure.



Primer vistazo - Operadores



- Operadores más destacados de proveedores:

- *Http*

HttpOperator	Calls an endpoint on an HTTP system to execute an action.
---------------------	---

- *Docker*

DockerOperator	Execute a command inside a docker container.
-----------------------	--

- *Hive*

HiveOperator	Executes hql code or hive script in a specific Hive database.
---------------------	---

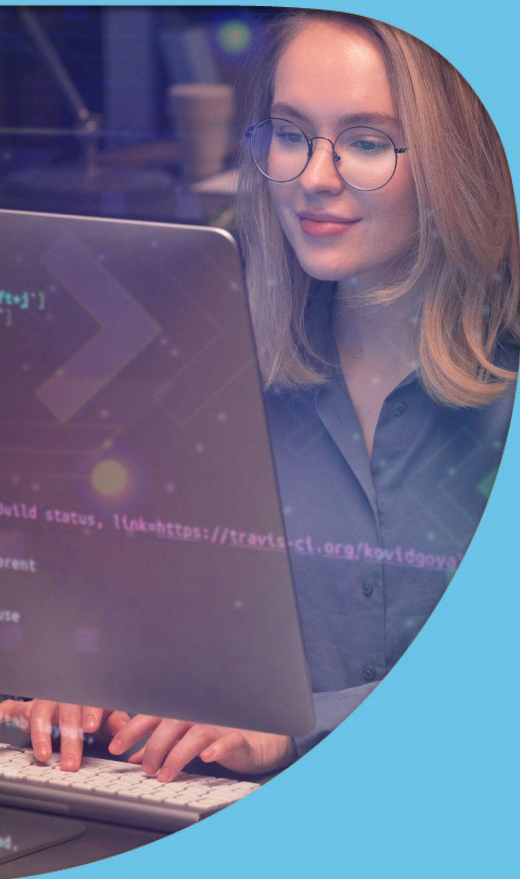


Primer vistazo - Operadores



- Operadores más destacados de proveedores:
 - BigQuery*

<code>BigQueryUIColors</code>	Hex colors for BigQuery	<code>BigQueryDeleteDatasetOperator</code>	Delete an existing dataset from your Project in BigQuery.
<code>IfExistAction</code>	Action to take if the resource	<code>BigQueryCreateEmptyDatasetOperator</code>	Create a new dataset for your Project in BigQuery.
<code>BigQueryCheckOperator</code>	Performs checks against	<code>BigQueryGetDatasetOperator</code>	Get the dataset specified by ID.
<code>BigQueryValueCheckOperator</code>	Perform a simple value	<code>BigQueryGetDatasetTablesOperator</code>	Retrieve the list of tables in the specified dataset.
<code>BigQueryIntervalCheckOperator</code>	Check that the values	<code>BigQueryUpdateTableOperator</code>	Update a table for your Project in BigQuery.
<code>BigQueryColumnCheckOperator</code>	Subclasses the SQLCol	<code>BigQueryUpdateDatasetOperator</code>	Update a dataset for your Project in BigQuery.
<code>BigQueryTableCheckOperator</code>	Subclasses the SQLTab	<code>BigQueryDeleteTableOperator</code>	Delete a BigQuery table.
<code>BigQueryGetDataOperator</code>	Fetch data and return	<code>BigQueryUpsertTableOperator</code>	Upsert to a BigQuery table.
<code>BigQueryCreateTableOperator</code>	Creates a new table in	<code>BigQueryUpdateTableSchemaOperator</code>	Update BigQuery Table Schema.
<code>BigQueryDeleteDatasetOperator</code>	Delete an existing dataset from your Project in BigQuery.	<code>BigQueryInsertJobOperator</code>	Execute a BigQuery job.



05_

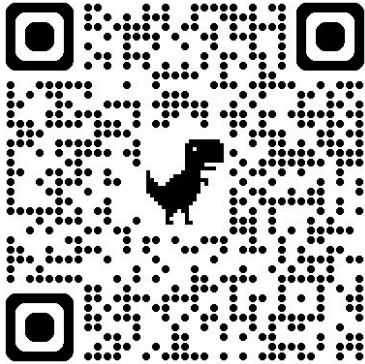
Toma de contacto

Manos a la obra

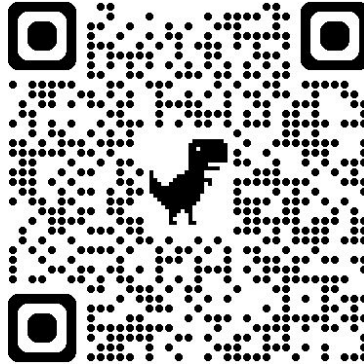




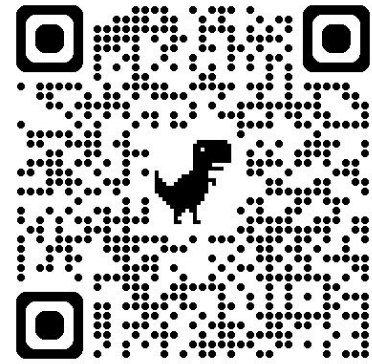
Toma de contacto



Homebrew



Instalación
Astronomer



Repositorio Github
con ejemplos

Gracias!



aulaupm@nextdigital.es

Orquesta tus datos, no tus dolores de cabeza

Anónimo

